



Documentação técnica

Base Operacional — arquitetura e integrações

Sistema	Base Operacional
Data	2 de agosto de 2026
Versão	1.0
Público	Desenvolvimento e manutenção

Sumário

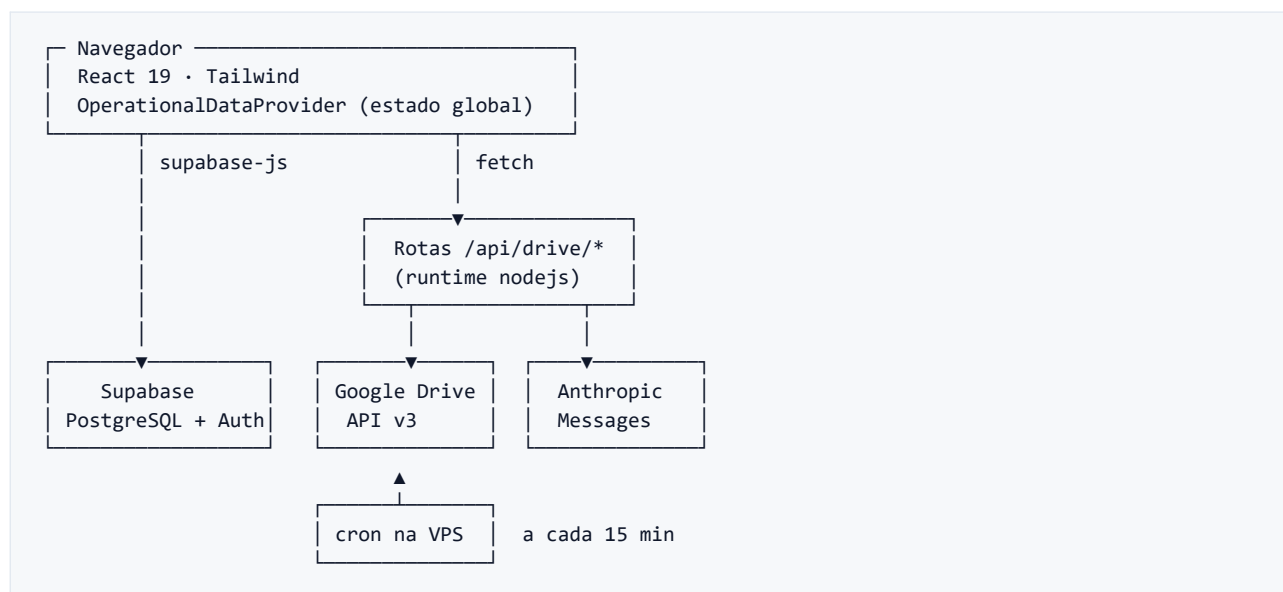
Visão geral	4
Dependências principais	4
Estrutura do projeto	5
Camadas	5
lib/ — domínio	5
services/ — acesso a dados	5
hooks/	6
Modelo de dados	6
clientes	6
processos	7
parceiros	7
historico_status	7
Tabelas de controle das rotinas	7
Domínio e mapeamento	8
Status	8
Datas	8
Valores monetários e percentuais	9
Nomes	9
Autenticação e autorização	9
Middleware	9
Dois níveis de proteção	9
Acesso ao banco	9
Estado no cliente	10
Composição dos dados	10
Escrita otimista com rollback por exceção	10
Histórico de status	10
Modo local	10
Integração com o Google Drive	10
Credenciais	10
Estrutura de pastas	11
Operações	11
Upload	11
Sincronização Drive → banco	12

Sincronização incremental — drive-sync-engine.ts	12
Varredura completa — drive-full-scan.ts	12
Trava — drive-scan-lock.ts	12
Leitura de documentos por IA	13
Deduplicação	13
Contexto	13
Classificação	13
Download e extração de texto	13
Extração	14
Sanitização	14
Vinculação	14
Criação de processo a partir de arquivo solto	14
Progresso em tempo real	15
Sincronização da planilha	15
Referência da API	15
Protegidas por sessão	15
Protegidas por x-sync-secret	16
Variáveis de ambiente	16
Build e deploy	17
Docker	17
Agendamento	17
Convenções de código	17
Pontos de atenção	18
Autorização depende de RLS	18
Status inicial de processo divergente	18
"Últimos cadastrados" do Dashboard não são os últimos	18
Contador de processos criados	18
ai-document-reader.ts concentra responsabilidades	18
Sem testes automatizados	18
Avisos de lint pendentes	18
/api/drive/upload sem uso na interface	18

Arquitetura, modelo de dados e integrações. Para o comportamento visto pelo usuário, veja o manual operacional.

Visão geral

Aplicação Next.js 16 (App Router) que roda como servidor Node. Não há backend separado: as rotas de API do próprio Next fazem a ponte com o Google Drive e com a API da Anthropic, enquanto o acesso ao PostgreSQL acontece via cliente Supabase tanto do navegador quanto do servidor.



Dependências principais

Pacote	Uso
<code>next 16.2.9</code>	Framework, App Router, rotas de API
<code>react / react-dom 19.2.7</code>	Interface
<code>@supabase/supabase-js</code>	Acesso ao PostgreSQL
<code>@supabase/ssr</code>	Sessão via cookies no middleware e no servidor
<code>googleapis</code>	Google Drive API v3
<code>@anthropic-ai/sdk</code>	Extração de dados de documentos
<code>mammoth</code>	Texto de <code>.docx</code> e <code>.odt</code>
<code>word-extractor</code>	Texto de <code>.doc</code> legado
<code>lucide-react</code>	Ícones
<code>tailwindcss 3.4</code>	Estilo

TypeScript em modo `strict`, com alias `@/*` apontando para a raiz.

Estrutura do projeto

app/	
(main)/	páginas autenticadas (dashboard, clientes, processos)
api/drive/	rotas de API – toda a integração com Drive e IA
login/	tela de login
layout.tsx	layout raiz
components/	componentes de interface
forms/	modais de cadastro e mudança de status
layout/	cabeçalho e container de página
ui/	botão e modal genéricos
hooks/	hooks de navegação, ações e upload no Drive
lib/	domínio puro, tipos, validação, clientes Supabase/Google
google/drive.ts	credenciais e cliente do Drive
supabase/	clientes para navegador, servidor e middleware
services/	acesso a dados e integrações
middleware.ts	proteção de rotas

A separação que importa: **lib/ não conhece rede**. São tipos, regras de domínio e funções puras — o que permite testar e reusar sem tocar em I/O. **services/** é onde mora tudo que fala com Supabase, Drive ou Anthropic.

Camadas

lib/ — domínio

Arquivo	Responsabilidade
types.ts	Tipos do domínio: Client , LegalProcess , Parceiro , status
domain.ts	Listas de status, classes dos selos, toTitleCase , normalizeText
client-queries.ts	Filtro combinado da lista e detecção de duplicidade
client-view-model.ts	Formatação para exibição (documento, rótulos, contagens)
date-utils.ts	Conversão entre YYYY-MM-DD e DD/MM/YYYY , cálculo de duração
validation.ts	E-mail, percentual, moeda e normalização de documento
factories.ts	Constrói Client/LegalProcess a partir dos formulários
drive-status-map.ts	Mapa pasta ↔ status e parsing de nomes de pasta
id.ts	Geração de id legível (cliente-maria-souza-1735...)
local-store.ts	Persistência em localStorage no modo sem Supabase

services/ — acesso a dados

Arquivo	Responsabilidade
clientes.ts / processos.ts / parceiros.ts	CRUD no Supabase
mappers.ts	Conversão entre linha do banco e tipo de domínio
historico-status.ts	Registro e leitura do histórico de status

Arquivo	Responsabilidade
<code>google-drive.ts</code>	Criação e movimentação de pastas
<code>drive-status-sync.ts</code>	Regras compartilhadas de vínculo pasta ↔ cadastro
<code>drive-sync-engine.ts</code>	Sincronização incremental (API de changes)
<code>drive-full-scan.ts</code>	Varredura completa com checkpoint
<code>drive-scan-lock.ts</code>	Trava contra varreduras simultâneas
<code>drive-sync-status.ts</code>	Data da última sincronização
<code>drive-sync.ts</code>	Chamadas do navegador para as rotas de pasta
<code>ai-document-reader.ts</code>	Pipeline de leitura de documento por IA
<code>sheets-sync.ts</code>	Importação da planilha de honorários

hooks/

Hook	Responsabilidade
<code>useDriveNavigation</code>	Navegação, breadcrumbs e listagem de arquivos
<code>useDriveActions</code>	Criar pasta e excluir arquivo
<code>useGoogleDriveUpload</code>	OAuth do usuário, upload e leitura do progresso via SSE

Modelo de dados

PostgreSQL no Supabase. Os nomes de coluna são em português, refletindo o domínio; a conversão para os tipos em inglês do código acontece em `services/mappers.ts`.

clientes

Coluna	Tipo	Observação
<code>id</code>	uuid/text	Chave primária
<code>nome</code>	text	Nome ou razão social
<code>nome_fantasia</code>	text	—
<code>tipo</code>	text	PF ou PJ
<code>cpf_cnpj</code>	text	Só dígitos, sem pontuação
<code>rg_ie</code>	text	—
<code>data_nascimento_abertura</code>	date	—
<code>estado_civil</code>	text	—
<code>status</code>	text	Slug: <code>ativo</code> , <code>audiencia</code> , <code>arquivado</code> , <code>cancelado</code> , <code>contratacao</code> , <code>dativo</code> , <code>parceiros</code> , <code>sarandi</code>
<code>telefone</code> , <code>email</code> , <code>endereço</code>	text	—

Coluna	Tipo	Observação
origem	text	—
parceiro_id	fk → <code>parceiros.id</code>	—
percentual_parceiro	numeric	—
data_cadastro, data_ativacao, data_finalizacao	date	—
drive_folder_id	text	Id da pasta no Drive
drive_path	text	Caminho legível (<code>02_ativos > maria_souza</code>)

Há restrição de unicidade em `cpf_cnpj` — o código trata a violação `23505` como "cliente já existe" e atualiza em vez de falhar.

processos

Coluna	Tipo	Observação
id	uuid/text	Chave primária
cliente_id	fk → <code>clientes.id</code>	—
numero_cnpj	text	"A definir" quando ainda não há número
parte_contraria, tipo_acao, vara_comarca	text	—
status	text	<code>ativo</code> , <code>contratacao</code> , <code>audiencia</code> , <code>arquivado</code> , <code>cancelado</code> e os legados
data_protocolo, data_encerramento	date	—
modelo_cobranca	text	—
valor_entrada	numeric	—
percentual_exito	numeric	—
localizacao	text	Projudi, Eproc PR, Eproc SP, Outro
anotacoes	text	—
drive_folder_id, drive_path	text	—

parceiros

`id`, `nome`, `percentual_honorario`.

historico_status

`id`, entidade (`cliente\|processo`), `entidade_id`, `status_anterior`, `status_novo`, `data_evento`, `observacao`.

Tabelas de controle das rotinas

Tabela	Chave	Colunas	Papel
<code>drive_sync_tokens</code>	<code>id = 'singleton'</code>	<code>page_token, updated_at</code>	Cursor da API de changes; <code>updated_at</code> alimenta o "sincronizado em" da interface
<code>drive_scan_checkpoint</code>	<code>id = 'singleton'</code>	<code>status_folder_id, cliente_folder_id, updated_at</code>	Onde a varredura completa parou
<code>drive_scan_lock</code>	<code>id = 'singleton'</code>	<code>locked, locked_at</code>	Trava contra execução paralela
<code>documentos_processados</code>	<code>file_id</code>	<code>modified_time, processed_at</code>	Evita reprocessar arquivo inalterado

As três primeiras usam o padrão de linha única com `id = 'singleton'`, o que permite `upsert` sem condição de corrida.

Domínio e mapeamento

Status

`Status` reúne os cinco valores atuais. `ClientStatus` e `ProcessStatus` estendem esse conjunto com os legados de cada entidade — assim o compilador obriga a tratá-los em toda exibição, mas eles ficam fora das listas de cadastro.

Três mapas fazem a tradução:

Mapa	Origem → destino	Onde vive
<code>STATUS_FOLDER_MAP</code>	nome da pasta → <code>ClientStatus</code>	<code>lib/drive-status-map.ts</code>
<code>STATUS_DB_MAP</code>	<code>ClientStatus</code> → slug do banco	<code>lib/drive-status-map.ts</code>
<code>STATUS_FOLDER_TO_DB_MAP</code>	nome da pasta → slug, derivado dos dois acima	<code>lib/drive-status-map.ts</code>

`matchStatusFolderKey()` casa o nome real da pasta no Drive com a chave do mapa ignorando maiúsculas e espaços — necessário porque o Drive aceita `03_` cancelados com espaço.

O caso de Audiência merece atenção: em `STATUS_DB_MAP` ele cai no slug `ativo`, porque não existe pasta própria para audiência no Drive. Já `mapClientStatusToDb()`, usado quando o status vem do formulário, preserva `audiencia` como valor próprio. A exceção está isolada e comentada em `services/mappers.ts`.

Datas

O banco guarda `YYYY-MM-DD`; a interface exhibe `DD/MM/YYYY`. A conversão é centralizada:

Função	Direção
<code>asDisplayDate</code> (mappers)	banco → exibição
<code>asDbDate</code> (mappers)	exibição → banco
<code>toInputDate</code> (date-utils)	exibição → <code><input type="date"></code>
<code>formatInputDate</code> (date-utils)	input → exibição

Valores monetários e percentuais

Trafegam como string formatada no domínio ("R\$ 1.500,00", "20%") e viram `numeric` no banco via `parsePercentToNumber` e do parsing em `mapProcessoToDb`. Campos vazios viram "-" na exibição e `null` no banco.

Nomes

`toTitleCase` normaliza nomes gritados (MARIA SOUZA) ou todos em minúsculas para Title Case, mantendo preposições minúsculas. Nomes que já têm capitalização mista são preservados, para não estragar Construtora Vega Ltda..

Autenticação e autorização

Supabase Auth com e-mail e senha. Não há cadastro pela aplicação: os usuários são criados no painel do Supabase.

Middleware

`middleware.ts` intercepta tudo exceto assets estáticos. O fluxo:

1. Rotas públicas passam direto: `/login` e as cinco rotas de automação que têm segredo próprio (`/api/drive/sync`, `/api/drive/ai-read`, `/api/drive/scan-full`, `/api/drive/sheets-sync`, `/api/drive/processos/criar-pastas`).
2. Para o resto, `getSessionUser()` revalida a sessão.
3. Sem sessão: requisição a `/api/*` recebe 401; navegação é redirecionada para `/login?next=<rota>`.

A validação usa `supabase.auth.getUser()`, não `getSession()` — `getSession()` só lê o cookie local, que pode ser adulterado no cliente.

Dois níveis de proteção

Nível	Rotas	Como protege
Sessão	Páginas e rotas de API chamadas pela interface	Cookie de sessão do Supabase, validado no middleware
Segredo	Rotas de automação	Header <code>x-sync-secret</code> conferido contra <code>DRIVE_SYNC_SECRET</code>

Acesso ao banco

`lib/supabase/client.ts` cria um cliente com a **chave anônima**, usado tanto pelos módulos chamados do navegador quanto pelas rotas de servidor. Não há uso de service role key. Consequência prática: **a autorização efetiva sobre os dados depende das políticas de RLS configuradas no Supabase** — a proteção do middleware cobre o acesso à aplicação, não o acesso direto à API do Supabase com a chave anônima, que é pública por natureza.

Estado no cliente

`components/OperationalDataProvider.tsx` é o contexto único de dados operacionais. Ele carrega clientes, processos e parceiros de uma vez no mount e mantém tudo em memória; as telas leem daí em vez de refazer consultas.

Composição dos dados

Ao carregar, duas funções de `lib/factories.ts` completam o que o banco não entrega pronto:

- `attachProcessIds` — preenche `processIds` de cada cliente cruzando com a lista de processos;
- `attachPartnerNames` — resolve `parceiro_id` para o nome do parceiro.

Escrita otimista com rollback por exceção

Cada operação de escrita (`createClient`, `updateClientStatus`, ...) segue o mesmo formato: grava no Supabase, sincroniza a pasta no Drive, atualiza o estado local e mostra um toast. Falha vira mensagem de erro e `throw`, para o modal saber que não deve fechar.

Falhas de Drive são tratadas à parte, em `services/drive-sync.ts`: a função registra o erro e devolve o registro original, sem propagar. Um Drive indisponível não impede o cadastro — apenas deixa o registro sem pasta.

Histórico de status

`tryRegisterStatusHistory` só grava quando o status realmente mudou e nunca propaga erro: falhar ao registrar histórico não pode desfazer uma mudança de status já persistida. Cada gravação incrementa `statusHistoryVersion`, que `StatusHistoryList` observa para recarregar.

Modo local

Quando `NEXT_PUBLIC_SUPABASE_URL` ou `NEXT_PUBLIC_SUPABASE_ANON_KEY` não estão definidas, `isSupabaseConfigured` é `false` e o provider passa a ler e gravar em `localStorage`, sob a chave `base-operacional-boos:v2`. Serve para trabalhar na interface sem tocar na base real. Nesse modo não há sincronização com o Drive nem histórico de status.

Integração com o Google Drive

Credenciais

Autenticação por **service account** (JWT), escopo `https://www.googleapis.com/auth/drive`. A pasta raiz do escritório precisa estar compartilhada com o e-mail da service account.

`lib/google/drive.ts` concentra o acesso:

Função	Papel
<code>isGoogleDriveConfigured()</code>	Confere se as três variáveis existem
<code>validateGooglePrivateKey()</code>	Valida o formato e converte <code>\n</code> literais em quebras reais
<code>getGoogleDriveClient()</code>	Cliente autenticado

Função	Papel
<code>getGoogleDriveRootFolderId()</code>	Id da raiz, com erro claro se ausente

O tratamento do `\n` existe porque a chave privada normalmente é injetada como variável de ambiente em uma linha só.

Estrutura de pastas

```

raiz
├── 0N_status ← STATUS_FOLDER_MAP
│   ├── nome_do_cliente
│   │   ├── documentos_pessoais
│   │   ├── comunicacao
│   │   └── numero_cnj_tipo_acao
│   │       ├── inicial
│   │       └── peticoes_subsequentes

```

Nomes de pasta passam por `sanitizeFolderName`: sem acento, minúsculas, underscore no lugar de qualquer caractere não alfanumérico.

Operações

Função (services/google-drive.ts)	O que faz
<code>criarPastaCliente</code>	Cria a pasta no status correspondente + subpastas padrão
<code>criarPastaProcesso</code>	Cria a pasta do processo dentro da do cliente + subpastas
<code>moverPastaClientePorStatus</code>	Move a pasta entre pastas de status (<code>addParents/removeParents</code>)
<code>findOrCreateFolder</code>	Idempotente: procura por nome no pai antes de criar

`findOrCreateFolder` ser idempotente é o que permite reexecutar sincronizações sem multiplicar pastas.

Valores interpolados na query do Drive passam por `escapeQueryValue`, que escapa barras e aspas simples.

Upload

Dois caminhos, com finalidades diferentes:

Caminho	Credencial	Usado por
Navegador → API do Google	OAuth do próprio usuário (<code>drive.file</code>)	Botão de upload do navegador de arquivos
Navegador → <code>/api/drive/upload</code>	Service account	Nenhuma tela hoje; endpoint disponível para automação

O caminho pelo navegador usa Google Identity Services. Um detalhe não óbvio está em `useGoogleDriveUpload.openFilePicker`: o token é renovado **antes** de abrir o seletor de arquivos, ainda dentro do clique do usuário. Se a renovação ficasse para depois, o navegador bloquearia o popup do Google silenciosamente — há um comentário no código registrando isso.

A rota `/api/drive/upload` aceita apenas PDF e imagens, limita a 20 MB e sanitiza o nome do arquivo contra path traversal.

Sincronização Drive → banco

Duas estratégias sobre a mesma base de regras, extraída em `services/drive-status-sync.ts`:

Função	Papel
<code>findClienteByDriveFolderId</code>	Cliente já vinculado à pasta
<code>linkOrCreateClienteFromFolder</code>	Vincula ao cliente existente de mesmo nome ou cria um novo
<code>updateClienteStatusFromFolder</code>	Atualiza status e caminho a partir da pasta
<code>findProcessoByDriveFolderId / createProcessoFromFolder</code>	Equivalentes para processo

A regra de `linkOrCreateClienteFromFolder` merece destaque: antes de criar, ela procura um cliente com o mesmo nome. Se achar e ele ainda não tiver pasta, apenas vincula. É o que impede a recriação de pasta no Drive de gerar cadastro duplicado.

Sincronização incremental — `drive-sync-engine.ts`

Usa a API de changes do Drive com cursor persistido:

1. Lê `page_token` de `drive_sync_tokens`. **Se não existir**, grava o `startPageToken` atual e encerra — a primeira execução só estabelece a linha de base, sem processar histórico.
2. Percorre as páginas de mudanças. Para cada item não removido e não na lixeira:
 - **arquivo** → passa por `readDriveFile` (leitura por IA);
 - **pasta dentro de pasta de status** → cria/vincula cliente ou atualiza status;
 - **pasta dentro de pasta de cliente** → cria processo.
1. Ao receber `newStartPageToken`, grava o novo cursor e encerra.

Erros por item são acumulados em `result.errors` sem interromper o restante.

Varredura completa — `drive-full-scan.ts`

Percorre a árvore inteira: raiz → pastas de status → clientes → processos e arquivos.

O limite de 5 minutos de execução é tratado com checkpoint. `MAX_DURATION_MS` é 4,5 minutos; ao ultrapassar, a rotina grava em `drive_scan_checkpoint` a última pasta de cliente **totalmente concluída** e sai. A execução seguinte pula tudo até aquele ponto e retoma dali. Ao concluir uma volta inteira sem estourar o tempo, o checkpoint é limpo — assim a próxima rodada revisita tudo e pega pastas antigas que ganharam arquivo novo.

Trava — `drive-scan-lock.ts`

Ambas as rotinas adquirem a mesma trava antes de rodar. A aquisição é um `UPDATE` condicional (`locked = false` ou `locked_at` mais antigo que 10 minutos) com `select()` — se nenhuma linha voltar, outra execução está em andamento e a rota responde `409`. A condição de idade garante que uma execução travada não bloqueie o sistema para sempre.

Leitura de documentos por IA

Pipeline em `services/ai-document-reader.ts`, entrada por `readDriveFile()`.

```
verifica se já foi lido
↓
identifica contexto (pasta do cliente? do processo?)
↓
classifica o tipo de documento
↓
baixa e extrai texto (ou manda binário)
↓
extrai campos com Claude
↓
sanitiza os valores
↓
localiza cliente/processo
↓
grava e marca como processado
```

Deduplicação

Compara `modifiedTime` do Drive com `modified_time` em `documentos_processados`. Iguais → pula tudo. É o que impede a varredura completa de reprocessar e recobrar a API a cada volta.

Contexto

Pelo pai e pelo avô da pasta:

Situação	Resultado
Arquivo direto na pasta de status	Pulado — não pertence a nenhum cliente
Pai é pasta de cliente	Cliente identificado
Pai é pasta de processo já cadastrada	Processo e cliente identificados
Pai é subpasta do cliente	Cliente é o avô; contexto vira <code>documentos_pessoais/</code>

Classificação

`detectDocumentType()` usa nome do arquivo e caminho, sem chamar a IA: `procuracao`, `contrato_honorarios`, `documento_pessoal`, `documento_inicial` ou `outro`. O tipo escolhe o prompt.

Download e extração de texto

Formato	Tratamento
PDF, JPEG, PNG, GIF, WEBP	Enviado direto ao modelo, em base64 (limite 20 MB)
Documentos Google	Exportado como PDF
Planilhas Google	Exportado como CSV
<code>.docx</code> , <code>.odt</code>	Texto via <code>mammoth</code>
<code>.doc</code>	Texto via <code>word-extractor</code>

Formato	Tratamento
.rtf	Remoção de comandos RTF por regex
text/*, JSON, XML	Texto direto
ZIP, RAR, vídeo, áudio, binários	Pulado

Extração

Modelo `claude-haiku-4-5-20251001`, `max_tokens` 1024, um prompt por tipo de documento pedindo JSON. Todos os prompts carregam a mesma instrução: **campo não encontrado deve voltar null**, nunca "não informado", "N/A" ou similar.

A resposta é lida com regex de primeiro objeto JSON e `JSON.parse` protegido — resposta fora do formato vira objeto vazio, não exceção.

Sanitização

`sanitizeValue()` é a defesa contra o modelo:

- descarta placeholders por regex (`não informado`, `n/a`, `desconhecido`, `-`, `null`, ...), mesmo tendo pedido `null` no prompt;
- datas: aceita `YYYY-MM-DD` e `DD/MM/YYYY`, converte para ISO e **descarta** qualquer outro formato, para não quebrar o `date` do Postgres;
- `cpf_cnpj`: reduz a dígitos, garantindo que a mesma pessoa não entre duas vezes por diferença de pontuação.

Campos descartados não entram no `UPDATE` — um dado ruim nunca sobrescreve um dado bom já existente.

Vinculação

Em ordem de confiança:

1. Processo pelo `drive_folder_id` da pasta pai;
2. Cliente pelo `drive_folder_id`;
3. Cliente pelo nome da pasta (`ilike` exato e, se falhar, `%primeiro%último%`) — ao achar, grava o `drive_folder_id` para as próximas leituras irem pelo caminho 2;
4. Criação de cliente novo, com o status derivado da pasta.

Na criação, violação de unicidade `23505` com CPF/CNPJ presente é tratada como "cliente já existe": busca pelo documento e atualiza o existente sem sobrescrever a pasta já vinculada a ele.

Criação de processo a partir de arquivo solto

`ensureProcessoFromClienteFile()` roda quando um documento inicial ou contrato de honorários está solto na pasta do cliente:

1. Com CNJ válido → procura processo do cliente com aquele número e atualiza;
2. Sem CNJ → prefere um processo sem `modelo_cobranca` definido; senão, qualquer um do cliente;
3. Nenhum processo existente → **só cria se for petição inicial**. Contrato de honorários sozinho não cria processo, espera a inicial.

Ao criar, tenta criar também a pasta no Drive. Falha aí é registrada e ignorada: o processo permanece salvo, apenas sem pasta.

Progresso em tempo real

`readDriveFile` aceita um `ProgressEmitter` opcional. A rota `/api/drive/read-file` monta um `ReadableStream` que traduz cada evento em SSE (`data: {...}\n\n`), consumido por `useGoogleDriveUpload` para alimentar a linha do tempo do upload. Chamadas sem emitter (varreduras) simplesmente não emitem.

Sincronização da planilha

`services/sheets-sync.ts` exporta a planilha de honorários como CSV e cruza com a base.

O parser de CSV é próprio — respeita aspas e vírgulas dentro de campo. As colunas são referenciadas por `SHEET_COLUMN`: cliente (0), CNJ (1), honorários do escritório (3), honorários de terceiro (4) e indicação (7).

Para cada linha:

1. Localiza o processo pelo CNJ. Se `numero_cnj` estava como "A definir", completa; atualiza o percentual de êxito.
2. Não achando pelo CNJ, tenta o cliente pelo nome normalizado e, se falhar, por correspondência parcial das duas primeiras palavras.
3. Com o cliente identificado, resolve o parceiro pelo nome — criando-o se não existir — e grava `parceiro_id` e `percentual_parceiro`.

Processos e clientes são carregados de uma vez e indexados em `Map`, evitando uma consulta por linha.

Referência da API

Todas as rotas rodam com `runtime = "nodejs"`.

Protegidas por sessão

Rota	Método	Descrição
<code>/api/drive/arquivos?folderId=</code>	GET	Lista arquivos e pastas
<code>/api/drive/arquivos/[id]</code>	DELETE	Exclui arquivo
<code>/api/drive/pastas</code>	POST	Cria pasta (<code>name</code> , <code>parentId</code>)
<code>/api/drive/upload</code>	POST	Upload via service account (multipart; PDF/imagem, ≤20 MB)
<code>/api/drive/clientes</code>	POST	Cria a pasta do cliente e grava id/caminho
<code>/api/drive/clientes/status</code>	PATCH	Move a pasta do cliente para o novo status
<code>/api/drive/processos</code>	POST	Cria a pasta do processo
<code>/api/drive/read-file</code>	POST	Leitura por IA com progresso via SSE
<code>/api/drive/health</code>	GET	Diagnóstico da conexão com o Drive

`/api/drive/health` traduz falhas comuns em mensagem acionável: chave mal formatada, id de pasta inválido, service account sem acesso à raiz.

Protegidas por `x-sync-secret`

Rota	Método	maxDuration	Descrição
<code>/api/drive/sync</code>	POST	padrão	Sincronização incremental
<code>/api/drive/scan-full</code>	POST	300 s	Varredura completa
<code>/api/drive/ai-read</code>	POST	padrão	Lê um arquivo específico, sem streaming
<code>/api/drive/sheets-sync</code>	POST	120 s	Importa a planilha
<code>/api/drive/processos/criar-pastas</code>	POST	120 s	Cria pastas para processos sem pasta

Exemplo de chamada:

```
curl -X POST https://SEU_DOMINIO/api/drive/sync -H "x-sync-secret: $DRIVE_SYNC_SECRET"
```

`/api/drive/sync` e `/api/drive/scan-full` respondem `409` quando já existe uma varredura em andamento.

Variáveis de ambiente

Variáveis com prefixo `NEXT_PUBLIC_` são embutidas no bundle e ficam **visíveis no navegador** — nunca coloque segredo nelas.

Variável	Escopo	Descrição
<code>NEXT_PUBLIC_SUPABASE_URL</code>	público	URL do projeto Supabase
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	público	Chave anônima
<code>NEXT_PUBLIC_DRIVE_ROOT_FOLDER_ID</code>	público	Raiz usada pelo navegador de arquivos
<code>NEXT_PUBLIC_GOOGLE_CLIENT_ID</code>	público	Client id OAuth para o upload pelo navegador
<code>NEXT_PUBLIC_GOOGLE_UPLOAD_ACCOUNT_HINT</code>	público	Sugere a conta Google na tela de autorização
<code>GOOGLE_CLIENT_EMAIL</code>	servidor	E-mail da service account
<code>GOOGLE_PRIVATE_KEY</code>	servidor	Chave privada (aceita <code>\n</code> literais)
<code>GOOGLE_DRIVE_ROOT_FOLDER_ID</code>	servidor	Raiz usada pelas rotinas
<code>GOOGLE_SHEETS_ID</code>	servidor	Planilha de honorários
<code>DRIVE_SYNC_SECRET</code>	servidor	Segredo das rotas de automação
<code>ANTHROPIC_API_KEY</code>	servidor	Chave da API Anthropic

Sem as variáveis do Supabase a aplicação sobe em modo local. Sem as do Drive, as rotas respondem `503` com mensagem explicando a configuração ausente.

Build e deploy

Docker

O `Dockerfile` parte de `node:20-alpine`, instala dependências, injeta as variáveis `NEXT_PUBLIC_*` como build args e roda `next build`. O container expõe a porta 3000.

As `NEXT_PUBLIC_*` precisam existir **no build**, não só em runtime — o Next as embute no bundle. As demais são lidas em runtime e devem ser passadas na execução do container.

```
docker build \
  --build-arg NEXT_PUBLIC_SUPABASE_URL=... \
  --build-arg NEXT_PUBLIC_SUPABASE_ANON_KEY=... \
  --build-arg NEXT_PUBLIC_DRIVE_ROOT_FOLDER_ID=... \
  --build-arg NEXT_PUBLIC_GOOGLE_CLIENT_ID=... \
  -t base-operacional .
```

Agendamento

As rotinas são disparadas por cron na VPS (antes eram workflows do GitHub Actions). A sincronização incremental roda a cada 15 minutos — o mesmo intervalo anunciado no cabeçalho da interface, que lê `drive_sync_tokens.updated_at`.

```
* /15 * * * * curl -fsS -X POST https://SEU_DOMINIO/api/drive/sync -H "x-sync-secret: SEGREDO"
```

Varredura completa e sincronização da planilha são acionadas sob demanda, ou em agendamento próprio mais espaçado.

Convenções de código

- **Idioma:** domínio e interface em português; tipos e funções puras em `lib/` em inglês (`Client`, `LegalProcess`). Nomes de coluna acompanham o banco, em português. As funções de `services/` que refletem operações do domínio também ficam em português (`criarCliente`, `listarProcessos`).
- **Comentários:** reservados para o "porquê" não óbvio — bloqueio de popup, exceção da Audiência, motivo do checkpoint. O "o quê" fica no nome.
- **Erros:** mensagens em português, voltadas a quem vai ler. Rotinas em lote acumulam erros por item e seguem; operações interativas propagam para o toast.
- **Estilo:** Tailwind direto no JSX. Paleta `navy` e sombra `soft` no `tailwind.config.ts`.
- **Sem ponto e vírgula final em objetos multilinha:** o projeto não usa vírgula pendente (trailing comma).

Antes de commitar:

```
npm run lint
```

```
npm run build
```

Pontos de atenção

Itens conhecidos, registrados para quem for mexer no código.

Autorização depende de RLS

Todo acesso ao banco usa a chave anônima. As políticas de RLS no Supabase são a única barreira real sobre os dados — o middleware protege a aplicação, não a API do Supabase. Vale auditar as políticas antes de qualquer mudança de escopo.

Status inicial de processo divergente

`drive-full-scan.ts` e `drive-sync-engine.ts` criam processos com status `em_andamento` (que a interface exibe como o legado "Andamento"), enquanto `ai-document-reader.ts` cria com `ativo`. Convergir para `ativo` seria o esperado, mas mexer nisso exige decidir o que fazer com os registros já criados.

"Últimos cadastrados" do Dashboard não são os últimos

`DashboardView` monta as listas com `[...clients].slice(-6).reverse()`, mas `listarClientes()` ordena por `nome` e `listarProcessos()` por `numero_cnj` — os "últimos" são os finais do alfabeto, não os mais recentes. O comentário no código assume o id como proxy de ordem de criação, o que não se sustenta com essa ordenação. Corrigir exige ordenar por `data_cadastro` (clientes) e por uma coluna de criação nos processos, que hoje não existe.

Contador de processos criados

Em `runDriveSync`, `processosCreated` é incrementado após chamar `criarProcessoAutomatico`, que retorna sem criar nada quando o cliente da pasta pai não existe no banco. O número relatado pode ficar acima do real. Afeta apenas o relatório, não os dados.

`ai-document-reader.ts` concentra responsabilidades

São ~870 linhas cobrindo download, prompts, sanitização, vinculação e persistência. É o arquivo mais crítico do fluxo de IA e o candidato natural a ser dividido — sem testes automatizados, a divisão pede cuidado.

Sem testes automatizados

A verificação hoje é `lint` + `build` + conferência manual. Não há suíte de testes. As funções puras de `lib/` (`date-utils`, `validation`, `client-queries`, `drive-status-map`) são o ponto de partida mais barato, por não dependerem de rede.

Avisos de lint pendentes

`useDriveNavigation.ts` e `useGoogleDriveUpload.ts` acusam `react-hooks/set-state-in-effect`. Funcionam, mas destoam do padrão recomendado pelo React 19.

`/api/drive/upload` sem uso na interface

A rota existe e funciona, mas nenhuma tela a chama — o upload do navegador de arquivos vai direto à API do Google com o OAuth do usuário. Ela ficou de uma abordagem anterior (revertida) e segue disponível para automação.